

ATIVIDADE PRÁTICA GUIADA

Biblioteca da Escola

Express · Handlebars · MVC | Projeto app_escola | Valor: 10 pontos

Aluno(a): _____ Turma: _____ Data: ____/____/____

O que você vai construir

A biblioteca da escola precisa de uma página onde a bibliotecária cadastre livros e veja o acervo em uma tabela, com indicação de quais títulos ainda têm exemplar disponível.

Diferente das atividades anteriores, aqui **nada está pronto**. Você vai criar a classe, o controller, a rota, registrar tudo no app.js e escrever a view. Use as páginas de alunos, que já estão funcionando no projeto, como referência sempre que travar.

Arquivos desta atividade

Arquivo	Ação	O que ele faz
src/controllers/livroController.js	criar	Classe Livro e os métodos index e store.
src/routes/livros.js	criar	Liga os endereços aos métodos do controller.
src/views/livros.handlebars	criar	A página que a bibliotecária vê.
src/app.js	alterar	Registrar a nova rota na aplicação.

**DICA**

Trabalhe na ordem das etapas. Cada etapa termina com um checkpoint: só avance depois de confirmar que ele passou.

Etapa 1 — A classe Livro (2 pontos)

Crie o arquivo `src/controllers/livroController.js` e comece pela classe. Ela representa um livro do acervo.

Campos obrigatórios

Campo	Tipo	Observação
codigo	número	Gerado automaticamente, começando em 1. Nunca vem do formulário.
titulo	texto	Recebido do formulário.
autor	texto	Recebido do formulário.
ano	número	Recebido do formulário. Precisa ser convertido.
exemplares	número	Quantos exemplares a biblioteca possui.
emprestados	número	Começa sempre em 0 quando o livro é cadastrado.

Esqueleto para completar

src/controllers/livroController.js

```
class Livro {
  static codigoGeral = 0;

  constructor(titulo, autor, ano, exemplares) {
    this.codigo = _____; // some 1 antes de atribuir
    this.titulo = _____;
    this.autor = _____;
    this.ano = _____; // converta para número
    this.exemplares = _____; // converta para número
    this.emprestados = _____;
  }
}

const livros = [];
```

⚠️ ATENÇÃO

Tudo que chega de um formulário HTML é texto, mesmo em um campo do tipo number. Se você guardar ano e exemplares sem converter, a soma do acervo vai concatenar em vez de somar: 5 + 3 daria 53. A conversão se faz com `Number(valor)` na hora de atribuir.

💡 DICA

Escrever `++Livro.codigoGeral` incrementa primeiro e usa o valor já incrementado, então o primeiro livro recebe o código 1. Escrever `Livro.codigoGeral++` faria o primeiro livro receber 0.

Etapa 2 — O controller (3 pontos)

No mesmo arquivo, abaixo da classe, crie a classe do controller com dois métodos.

Método store — cadastrar

Recebe os quatro campos do formulário, cria um objeto Livro, coloca no array e redireciona de volta para a listagem.

src/controllers/livroController.js (continuação)

```
store(req, res) {
  const { titulo, autor, ano, exemplares } = req.body;
  livros.push(new Livro(_____, _____, _____, _____));
  return res.redirect("/livros");
}
```

Método index — listar

Aqui está o ponto principal da atividade. A view **não pode calcular nada**, então é o controller que precisa produzir três informações novas para cada livro, além das que já existem.

Campo criado	Tipo	Como calcular
disponiveis	número	exemplares menos emprestados.
disponivel	verdadeiro ou falso	Verdadeiro quando ainda há pelo menos um exemplar livre.
situacao	texto	"Disponível" quando houver exemplar livre; "Todos emprestados" quando não houver.

Além da lista, o index precisa enviar dois totais para o cabeçalho da página: a quantidade de títulos e a soma de todos os exemplares do acervo.

src/controllers/livroController.js (continuação)

```
index(req, res) {
  const lista = livros.map((l) => ({
    ...l,
    disponiveis: _____,
    disponivel: _____,
    situacao: _____,
  }));

  return res.render("livros", {
    livros: lista,
    total: _____,
    totalExemplares: _____,
  });
}
```

DICA

Os três pontos em ...l copiam todos os campos originais do livro para o novo objeto. Sem eles, você perderia título, autor e os demais.

Para somar os exemplares de todos os livros, pesquise o método reduce de arrays. Também é aceitável resolver com um laço for.

Não esqueça a última linha do arquivo:

```
export default new LivroController();
```

✓ CHECKPOINT

- O arquivo tem a classe Livro, a constante livros e a classe LivroController com index e store.
- O arquivo termina com export default.
- O comando npm run lint não aponta erros neste arquivo.

Etapa 3 — A rota (1 ponto)

Crie o arquivo `src/routes/livros.js`. Ele associa cada endereço ao método correspondente do controller.

src/routes/livros.js

```
import { Router } from "express";
import livroController from "_____";

const router = Router();

router.get("/", _____);
router.post("/", _____);

export default router;
```

⚠ ATENÇÃO

O caminho do import precisa subir uma pasta e entrar em controllers, e a extensão .js é obrigatória. Sem ela o Node acusa ERR_MODULE_NOT_FOUND.

Escreva o método do controller sem parênteses. Com parênteses, a função rodaria uma vez na inicialização e a rota deixaria de funcionar.

Registrar no app.js

Acrescente o import junto aos demais, no topo do arquivo, e a linha dentro do método routes().

src/app.js (trecho)

```
import livroRoutes from "../routes/livros.js";

routes() {
  this.app.use("/alunos", alunoRoutes);
  this.app.use("/livros", _____);
}
```

✓ CHECKPOINT

Ao rodar npm run dev, o servidor sobe sem erro no terminal.

Ao acessar `http://localhost:3000/livros`, aparece o erro Failed to lookup view "livros". Isso é esperado: a view ainda não existe e será criada na próxima etapa.

Etapa 4 — A view (4 pontos)

Crie `src/views/livros.handlebars`. Os dados que chegam do controller são estes:

```
{
  livros: [
    { codigo, titulo, autor, ano, exemplares,
      disponiveis, disponivel, situacao },
    ...
  ],
  total: 3,
  totalExemplares: 12
}
```

4.1 Cabeçalho com resumo

Um título para a página e, abaixo, uma linha informando quantos títulos e quantos exemplares existem no acervo.

4.2 Formulário de cadastro

Método POST, action /livros, com quatro campos. Os nomes precisam ser exatamente estes, porque é por eles que o controller lê os dados:

Campo na tela	Tipo do input	Atributo name
Título	text	titulo
Autor	text	autor
Ano	number	ano
Exemplares	number, min 1	exemplares

4.3 Tabela do acervo

Uma linha por livro, com as colunas: código, título, autor, ano, disponíveis e situação. A coluna de disponíveis deve aparecer no formato “2 de 3”.

4.4 Situação com cor

Na última coluna, exiba o texto da situação em verde quando houver exemplar livre e em vermelho quando não houver. Use o campo booleano que o controller enviou.

4.5 Acervo vazio

Quando não houver nenhum livro, a tabela mostra uma única linha com a mensagem “Nenhum livro cadastrado”. Use o `{{else}}` do próprio `{{each}}`, e não um `{{if}}` separado.

Esqueleto para completar

```
src/views/livros.handlebars
```

```
<h2>Biblioteca</h2>
<p>{{_____}} títulos · {{_____}} exemplares</p>

<form action="_____" method="_____">
  <input type="text" name="_____" placeholder="Título" required />
  <!-- complete os outros três campos -->
  <button type="submit">Cadastrar</button>
</form>

<table>
  <tr>
    <th>Código</th><th>Título</th><th>Autor</th>
    <th>Ano</th><th>Disponíveis</th><th>Situação</th>
```

```

</tr>

{{#each _____}}
  <tr>
    <td>{{codigo}}</td>
    <!-- complete as colunas de título, autor e ano -->
    <td>{{_____}} de {{_____}}</td>
    <td>
      {{#if _____}}
        <span style="color: green">{{_____}}</span>
      {{else}}
        <span style="color: red">{{_____}}</span>
      {{/if}}
    </td>
  </tr>
{{_____}}
<tr><td colspan="6">Nenhum livro cadastrado</td></tr>
{{/each}}
</table>

```

⚠️ ATENÇÃO

Dentro do {{#each}} o contexto muda: escreva {{título}}, e não {{livros.título}}.

Use sempre duas chaves. As três chaves só existem no {{{body}}} do layout.

Regras da atividade

- Nenhuma conta, comparação ou operador pode aparecer dentro do arquivo `.handlebars`. Todo valor exibido chega pronto do controller.
- Os nomes das variáveis na view devem ser idênticos aos enviados pelo controller. Nome errado não gera erro: o valor simplesmente some da tela.
- O array vive na memória. A cada reinício do servidor os livros somem, e isso é esperado nesta etapa do curso.

Como testar antes de entregar

Rode `npm run dev`, acesse `http://localhost:3000/livros` e confira nesta ordem:

1. A página abre com a mensagem de acervo vazio e o resumo zerado.
2. Cadastre “Dom Casmurro”, de Machado de Assis, 1899, com 2 exemplares. Ele deve aparecer com código 1, “2 de 2” e situação verde.
3. Cadastre um segundo livro com 3 exemplares. O resumo passa a mostrar 2 títulos e 5 exemplares.
4. Abra o console do terminal e confirme que nenhum erro apareceu.



DICA

Para testar a situação em vermelho sem criar a funcionalidade de empréstimo, altere temporariamente o valor inicial de emprestados no construtor e recarregue a página. Lembre de voltar para 0 antes de entregar.

Critérios de correção

Critério	Pontos
Classe Livro com os seis campos e conversão numérica correta	2,0
Método store cadastrando e redirecionando	1,0
Método index criando disponíveis, disponível e situacao	1,5
Totais de títulos e exemplares corretos	0,5
Rota criada e registrada no <code>app.js</code>	1,0
Formulário com os quatro campos e nomes corretos	1,0
Tabela listando todos os livros nas colunas pedidas	1,5
Situação exibida com a cor certa usando <code>{{#if}}</code>	1,0
Mensagem de acervo vazio usando <code>{{else}}</code>	0,5



ATENÇÃO

Perde 1,0 ponto cada conta, comparação ou regra de negócio encontrada dentro de um arquivo `.handlebars`.

Desafio opcional (+1 ponto extra)

1. Extraia a linha da tabela para `src/views/partials/linhaLivro.handlebars` e chame-a de dentro do `each` com `{{> linhaLivro livro=this}}`.
2. Crie o botão “Emprestar”, que aumenta em 1 o campo `emprestados` do livro, respeitando o limite de exemplares.

Entrega

Apresente o projeto rodando ao professor e envie o link do repositório no Google Sala de Aula até ____/____.

Na apresentação, você deve saber responder:

Pergunta obrigatória

Por que o campo `situacao` foi criado no controller, se a própria view poderia comparar o número de disponíveis com zero?